

COMMUNICATION PROTOCOLS

Little Daisies

Naw , Br̄gr̄ , Ogar

Overview

→ Asynchronous communication

↳ UART

↳ RS-232

→ Synchronous communication

↳ SPI

↳ I²C

→ Comparison

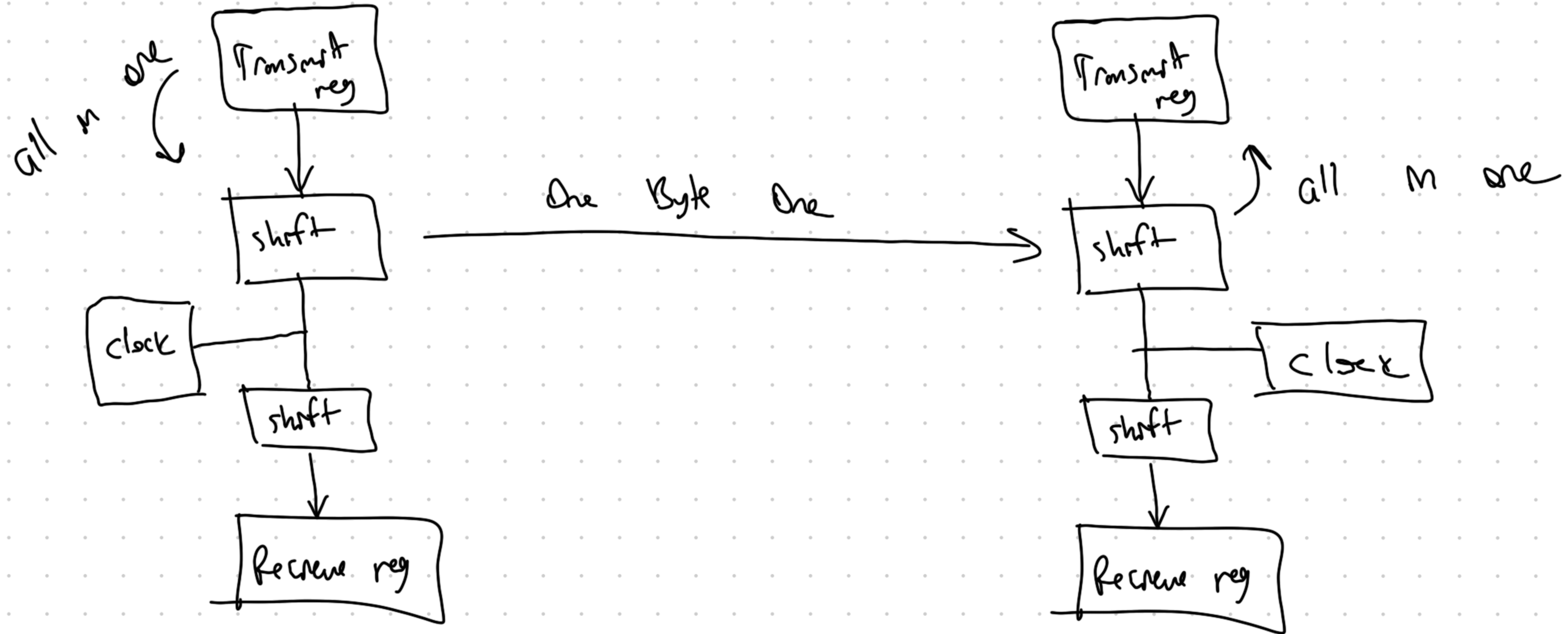
Asynchronous Communication.

- There is no need of clock synchronization.
- Data transmitted by baudrate, which is the time that they wait each other to send and receive data.
- Both transmitter and receiver has its own clock pulses, but they do not share with each other.

How to use those datas (Programming Approach):

1. Bit banging:
2. UART Peripheral approach: We have shift registers that also connected to the clocks. So all the data sent to the shift register from transmit register, then it will transmitted one byte one to the receiver's shift register.

Parallel n serial out $\longrightarrow$ Serial n Parallel out.



→ How to configure the UART communication:

- 1- Clock Configuration, according to the baud rate.
- 2- Data loading, in the transmit register.
- 3- Data Transmission, next step is to turn on the timer so it will start putting the data from the shift register to the transmission line and the other device will get the data according to the sampling rate.
- 4- Monitor data transmission,
 - ↳ 1. Looping method: We have to monitor the status bit which informs whether 8 bits are sent or not.
 - ↳ 2. Interrupt method: we do not need to monitor status continuously, because when the data is received or transmitted successfully, a particular interrupt is generated in the code to which an interrupt service routine will respond.

There are advantages of Interrupt method. It doesn't stick in the same place, we can complete the parallel actions in the system for eg we are sending the data from 1'st to 2'nd device, and in parallel we want to turn on some LEDs.

Advantages:

- 1 - Easy to interface
- 2 - Less hardware, only 2 wires
- 3 - Less software complications

Disadvantages:

- 1 - Synchronizing baud rates
- 2 - Only 2 devices
- 3 - No acknowledgement

Asynchronous Communication — UART

→ Universal Asynchronous Receiver Transmitter.

→ UART is character oriented protocol, that means data is sent byte by byte.

1. Bus topology:

→ In Bus topology, all the devices are connected to the bus wire and they communicate with each other via that bus. If a device needs to communicate with each other, they call other devices and communicate.

→ If one of the devices is disconnected, other devices can still be communicated with each other.

→ We use this type of topology in CAN protocol.

Asynchronous Communication - UART

2. Star Topology :

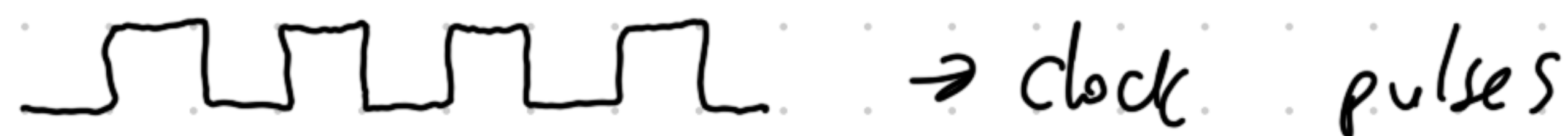
→ In this topology there is 1 master and all others are slaves. Master can communicate with all other slaves, but in the absence of master, there will be no communication.

3. AD-HOC / Peer to Peer Communication :

→ Where we need to communicate between 2 devices, this topology is used

→ There are no any kind of slave or master

→ UART is one of the protocols.

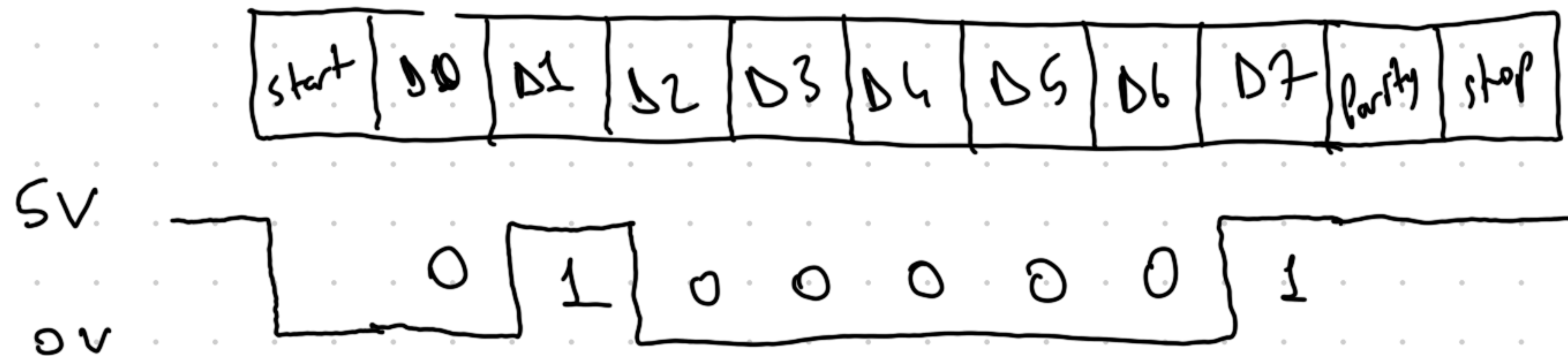


→ this is optional, some devices have some devices don't have

It is used only if the devices need to check the error present in the data stream.

→ In UART communication, initially both the Rx and Tx ports are high, indicating there is no data exchange.

→ If data started to be transmitted, the line is pulled down to low. After the starting byte, data starts to be transmitted.



→ For stop notation, the line stays high to notify the receiver.

It is 10-bit data; if you add a parity bit, it becomes 11-bit data.

→ In old devices, sometimes there were 2 stop signs due to slow devices to get ready for the next frame.

UART Example in Mechaboard

→ We send code from PC to Pi Pico using UART ports of our computer and picos micro usb port.

→ Also in serial connection, such as our scope example, we set the baudrate to 115000 to communicate between PC and Pico to transmit and receive PWM signals and modes, changes.

This is optional for our case due to the frequencies that we chose and the data transmit lines. We can use longer cables or shorter versions without any data loss.

Asynchronous Communication - RS-232

→ If the distance need to be too long, then the kind of communication can be effected by all kinds of electromagnetic interference from surrounding like high frequency waves of mobile phones which can distort 5V and 0V signal.

→ Due to distance and data loss, in earlier days RS-232 type of UART communication used to come into picture.

→ RS-232 needs 9 pin connector.

→ We will look at the actual data of the RS232 because it is clearly based on the UART communication.

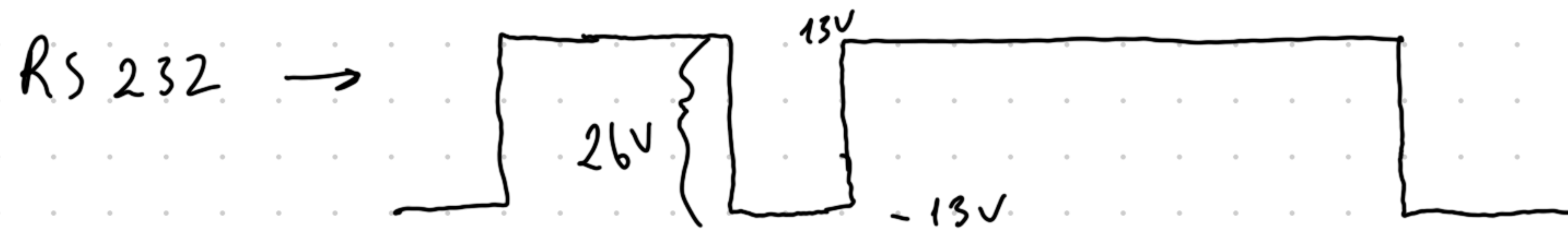
→ 1's and 0's are expressed in a unique way for eg

1's ranges from -3V to -25V

0's ranges from 3V to 25V

→ for example :

↳ if high "1" expressed as $-13V$ and low "0" expressed as $13V$
the actual representation of the signal is :



So in RS232 the amplitude is too high so that if data is corrupted due to any electromagnetic waves from surrounding, still 1's and 0's are clear so that data can be transmitted.

→ But the voltage levels are too high for an embedded system device so these voltages are converted into 0V to 5V level, by level shifter IC's.

Synchronous Communication.

- Devices which are communicating with each other share same clock pulses.
- Transmitter's data out is connected to receiver's data in. Also both of their clock pins are connected. If clock pulses are same, data will be received.
- After all data is sent, transmitter sends data saying all data transmitted. If receiver receives all the data, it also transmits all data gathered message.
- If transmitter does not get the acknowledgment message, it will resend the data.
- This cycle keeps repeating itself.

